457.557 Water Resources Systems Engineering, Spring 2022
Assignment #2

2020-29910 Hyeonju Kim

1. Solve and attach your solving procedures on the following problems on the textbook:

A. 4.1 (a), (b), (c), & (d)-Engineering economics
Consider two alternative water resource projects, A and B. Project A will cost $2,533,000 and will return $1,000,000 at the end of 5 years and $4,000,000 at the end of 10 years. Project B will cost $4,000,000 and will return $2,000,000 at the end of 5 and 15 years, and another $3,000,000 at the end of 10 years. Project A has a life of 10 years, and B has a life of 15 years. Assuming an interest rate of 0.1 (10%) per year:

(a) **What is the present value of each project?**

The present value, V_0, of an amount of money V_n at the end of period n is
$v_0 = V_n/(1+r)^n$
Present value = net present value - initial cost
Thus, present value for A is

$$v_0^P = \frac{1{,}000{,}000}{1.1^5} + \frac{4{,}000{,}000}{1.1^{10}} - 2{,}533{,}000 = -369{,}905.52(\$)$$

Present value for B is

$$V_0^P = \frac{2{,}000{,}000}{1.1^5} + \frac{3{,}000{,}000}{1.1^{10}} + \frac{2{,}000{,}000}{1.1^{15}} - 4{,}000{,}000$$
$$= -1{,}122{,}743.38(\$)$$

(b) **What is each project's annual net benefit?**

The average annual net benefit, v^p, is
$v^p = v_0^p[r(1+r)^{T_P}]/[(1+r)^{T_P} - 1]$

For A, $v^p = -369{,}905.52 \times \frac{0.1 \times 1.1^{10}}{1.1^{10}-1} = -60{,}200.42(\$)$

For B, $v^p = -1{,}122{,}743.38 \times \frac{0.1 \times 1.1^{15}}{1.1^{15}-1} = -147{,}611.31(\$)$

1-A(a), (b)

A			
initial cost	2,533,000		
r	0.1		V0=Vn/(1+r)^n
n(year)		(1+r)^n	Present value(V0)
1		1.100	
2		1.210	
3		1.331	
4		1.464	
5	1,000,000	1.611	620,921.32
6		1.772	
7		1.949	
8		2.144	
9		2.358	
10	4,000,000	2.594	1,542,173.16
	5,000,000		2,163,094.48
	present value-initial costs		- 369,905.52
	Average annual net benefit		- 60,200.42

B			
initial cost	4,000,000		
r	0.1		V0=Vn/(1+r)^n
n(year)		(1+r)^n	Present value(V0)
1		1.100	
2		1.210	
3		1.331	
4		1.464	
5	2,000,000	1.611	1,241,842.65
6		1.772	
7		1.949	
8		2.144	
9		2.358	
10	3,000,000	2.594	1,156,629.87
11		2.853	
12		3.138	
13		3.452	
14		3.797	
15	2,000,000	4.177	478,784.10
	7,000,000		2,877,256.61
	present value-initial costs		- 1,122,743.39
	Average annual net benefit		- 147,611.31

(c) **Would the preferred project differ if the interest rates were 0.05?**

The preferred project does not differ if the interest rates were 0.05. When the interest rate is 0.1, Project A's present value and annual net income are higher than Project B's, which is why it is preferred. At an interest rate of 0.05, the present value and annual net income of each project is:

Present value

For A, $v_0^P = \frac{1{,}000{,}000}{1.05^5} + \frac{4{,}000{,}000}{1.05^{10}} - 2{,}533{,}000 = 706{,}179.18(\$)$

For B, $v_0^P = \frac{2{,}000{,}000}{1.05^5} + \frac{3{,}000{,}000}{1.05^{10}} + \frac{2{,}000{,}000}{1.05^{15}} - 4{,}000{,}000 = 370{,}826.29(\$)$

The average annual net benefit, v^p, is

For A, $v^p = 706{,}179.18 \times \frac{0.05 \times 1.05^{10}}{1.05^{10}-1} = 91{,}453.43(\$)$

For B, $v^p = 370{,}826.29 \times \frac{0.05 \times 1.05^{15}}{1.05^{15}-1} = 35{,}726.25(\$)$

That is, project A is preferred over B because it is more profitable.

1-A(c)

A			
initial	2,533,000		
r	0.05		V0=Vn/(1+r)^n
n(year)		(1+r)^n	Present value(V0)
1		1.050	
2		1.103	
3		1.158	
4		1.216	
5	1,000,000	1.276	783,526.17
6		1.340	
7		1.407	
8		1.477	
9		1.551	
10	4,000,000	1.629	2,455,653.01
	5,000,000		3,239,179.18
	present value-initial costs		706,179.18
	Average annual net benefit		91,453.43

B			
initial	4,000,000		
r	0.05		V0=Vn/(1+r)^n
n(year)		(1+r)^n	Present value(V0)
1		1.050	
2		1.103	
3		1.158	
4		1.216	
5	2,000,000	1.276	1,567,052.33
6		1.340	
7		1.407	
8		1.477	
9		1.551	
10	3,000,000	1.629	1,841,739.76
11		1.710	
12		1.796	
13		1.886	
14		1.980	
15	2,000,000	2.079	962,034.20
	7,000,000		4,370,826.29
	present value-initial costs		370,826.29
	Average annual net benefit		35,726.25

(d) **Assuming that each of these projects would be replaced with a similar project having the same time stream of costs and returns, show that by extending each series of projects to a common terminal year (e.g., 30 years), the annual net benefits of each series of projects would be will be same as found in part (b).**

As calculated below, when we extend each series of projects to a common terminal year (30 years), the annual net benefits of each series of projects do not differ, just the same as found in part (b).

Present value

For A,

$$v_0^P = \frac{1{,}000{,}000}{1.1^5} + \frac{4{,}000{,}000}{1.1^{10}} + \frac{1{,}000{,}000}{1.1^{15}} + \frac{4{,}000{,}000}{1.1^{20}} + \frac{1{,}000{,}000}{1.1^{25}} + \frac{4{,}000{,}000}{1.1^{30}} - 2{,}533{,}000 - \frac{2{,}533{,}000}{1.1^{10}} - \frac{2{,}533{,}000}{1.1^{20}} = -567{,}504.21(\$)$$

For B,

$$v_0^P = \frac{2{,}000{,}000}{1.1^5} + \frac{3{,}000{,}000}{1.1^{10}} + \frac{2{,}000{,}000}{1.1^{15}} + \frac{2{,}000{,}000}{1.1^{20}} + \frac{3{,}000{,}000}{1.1^{25}} + \frac{2{,}000{,}000}{1.1^{30}} - 4{,}000{,}000 - \frac{4{,}000{,}000}{1.1^{15}} = -1{,}391{,}519.23(\$)$$

The average annual net benefit, v^p, is

For A, $v^p = -567{,}504.21 \times \frac{[0.1 \times 1.1^{30}]}{[1.1^{30}-1]} = -60{,}200.42(\$)$

For B, $v^p = -1{,}391{,}519.23 \times \frac{[0.1 \times 1.1^{30}]}{[1.1^{30}-1]} = -147{,}611.31(\$)$

1-A(d)

A			
initial cost	2,533,000		
r	0.1		V0=Vn/(1+r)^n
n(year)		(1+r)^n	Present value(V0)
1		1.100	
2		1.210	
3		1.331	
4		1.464	
5	1,000,000	1.611	620,921.32
6		1.772	
7		1.949	
8		2.144	
9		2.358	
10	4,000,000	2.594	1,542,173.16
11		2.853	
12		3.138	
13		3.452	
14		3.797	
15	1,000,000	4.177	239,392.05
16		4.595	
17		5.054	
18		5.560	
19		6.116	
20	4,000,000	6.727	594,574.51
21		7.400	
22		8.140	
23		8.954	
24		9.850	
25	1,000,000	10.835	92,296.00
26		11.918	
27		13.110	
28		14.421	
29		15.863	
30	4,000,000	17.449	229,234.21
	15,000,000		3,318,591.25
	present value - initial costs		567,504.21
	Average annual net benefit		60,200.42

B			
initial cost	4,000,000		
r	0.1		V0=Vn/(1+r)^n
n(year)		(1+r)^n	Present value(V0)
1		1.100	
2		1.210	
3		1.331	
4		1.464	
5	2,000,000	1.611	1,241,842.65
6		1.772	
7		1.949	
8		2.144	
9		2.358	
10	3,000,000	2.594	1,156,629.87
11		2.853	
12		3.138	
13		3.452	
14		3.797	
15	2,000,000	4.177	478,784.10
16		4.595	
17		5.054	
18		5.560	
19		6.116	
20	2,000,000	6.727	297,287.26
21		7.400	
22		8.140	
23		8.954	
24		9.850	
25	3,000,000	10.835	276,887.99
26		11.918	
27		13.110	
28		14.421	
29		15.863	
30	2,000,000	17.449	114,617.11
	14,000,000		3,566,048.97
	present value - initial costs		1,391,519.23
	Average annual net benefit		147,611.31

B. 4.9 (a) & (b)-Lagrange multiplier

Assume water can be allocated to three users. The allocation, xj, to each use j provides the following returns: $R(x_1) = (12x_1 - x_1^2)$, $R(x_2) = (8x_2 - x_2^2)$ and $R(x_3) = (18x_3 - 3x_3^2)$. Assume that the objective is to maximize the total return, F(X), from all three allocations and that the sum of all allocations cannot exceed 10.

(a) How much would each use like to have?

$R(x_1) = 12x_1 - x_1^2$

$R(x_2) = 8x_2 - x_2^2$

$R(x_3) = 18x_3 - 3x_3^2$

$F(x) = (12x_1 - x_1^2) + (8x_2 - x_2^2) + (18x_3 - 3x_3^2)$

Maximize $F(x)$

Subject to: $x_1 + x_2 + x_3 \leq 10$

$L = (12x_1 - x_1^2) + (8x_2 - x_2^2) + (18x_3 - 3x_3^2) - \lambda(x_1 + x_2 + x_3 - 10)$

$\frac{\partial L}{\partial x_1} = 12 - 2x_1 - \lambda = 0$

$\frac{\partial L}{\partial x_2} = 8 - 2x_2 - \lambda = 0$

$\frac{\partial L}{\partial x_3} = 18 - 6x_3 - \lambda = 0$

$\frac{\partial L}{\partial \lambda} = 10 - x_1 - x_2 - x_3 = 0$

$\rightarrow x_1 = 10 - x_2 - x_3$

$12 - 2(10 - x_2 - x_3) - \lambda = 12 - 20 + 2x_2 + 2x_3 - \lambda = -8 + 2x_2 + 2x_3 - \lambda = 0$

$8 - \lambda = 2x_2$

$-8 + (8 - \lambda) + 2x_3 - \lambda = 0$

$-2\lambda + 2x_3 = 0 \rightarrow x_3 = \lambda$

$18 - 6\lambda - \lambda = 0 \rightarrow \lambda = \frac{18}{7} = x_3$

$8 - 2x_2 - \frac{18}{7} = 0 \rightarrow x_2 = \frac{19}{7}$

$x_1 + \frac{19}{7} + \frac{18}{7} = 10 \rightarrow x_1 = \frac{33}{7}$

$\therefore x_1 = \frac{33}{7}, \; x_2 = \frac{19}{7}, \; x_3 = \frac{18}{7}, \; \lambda = \frac{18}{7}$

(b) **Show that at the maximum total return solution the marginal values, $\partial(R(x_j))/\partial x_j$, are each equal to the shadow price or Lagrange multiplier (dual variable) λ associated with the constraint on the amount of water available.**

When $x_1=33/7$, $x_2=19/7$, $x_3=18/7$, and $\lambda=18/7$,
boundary condition, x1+x2+x3=10, and
F(x)= 75.14 which is the maximum amount. λ is the marginal change in the objective function F(x), that is a change in the constraint associated with the constraint j. It is the slope of the objective function against value of 10. In this problem, since each net benefit function is concave, a maximum resulted.

C. 4.15- Dynamic programming (deterministic)

Consider a three-season reservoir operation problem. The inflows are 10, 50 and 20 in seasons 1, 2 and 3 respectively. Find the operating policy that minimizes the sum of total squared deviations from a constant storage target of 20 and a constant release target **of 25 in each of the three seasons. Develop a discrete dynamic programming model that considers only 4 discrete storage values: 0, 10, 20 and 30. Assume the releases cannot be less than 10 or greater than 40. Show how the model's recursive equations change depending on whether the decisions are the releases or the final storage volumes. Verify the optimal operating policy is the same regardless of whether the decision variables are the releases or the final storage volumes in each period. Which model do you think is easier to solve? How would each model change if more importance were given to the desired releases than to the desired storage volumes?**

$TSD_t(S_t, S_{t+1}, R_t)$ is equal to $[(20 - S_t)^2 + (25 - R_t)^2]$
$TSD_t(S_t, S_{t+1}, R_t)$ should be minimized.
S_t take on the discrete values 0, 10, 20, and 30, and R_t the discrete values of 10, 20, 30, and 40. Q_t the discrete values of 10, 50, and 20 in seasons, 1, 2, and 3 respectively.

At each stage, or season t, for each discrete state (St, Qt) we can compute the release R_t or equivalently the final storage volume S_{t+1}, that minimizes
$F_t^n(s_t, Q_t) = \text{Minimum}\{TSD_t(s_t, R_t, s_{t+1}) + F_{t+1}^{n-1}(s_{t+1}, Q_{t+1})\} for\ \ all\ \ 0 \le s_t \le 30$
which is recursive equation.

If the decision variable is the release, then the constraints on that release R_t are
$R_t \le S_t + Q_t$
$R_t \ge S_t + Q_t - 30\ \ (the\ \ capacity)$
And
$S_{t+1} = S_t + Q_t - R_t$

If the decision variable is the final storage volume, S_{t+1}, the constraints on that final storage volume are
$0 \le s_{t+1} \le 30$
$S_{t+1} \le S_t + Q_t$
And
$R_t = S_t + Q_t - S_{t+1}$

By developing a discrete dynamic programming model, optimal operating policy is the same regardless of whether the decision variables are the releases or the final storage volumes in each period.
It is easier to solve when the decision variable is the final storage volume. In the case of the final storage volume, the calculation process can be immediately known, but in the case of the release, the value is obtained by checking it once more.
If more importance were given to the desired releases than to the desired storage volumes, multiply the weight, which is the relative importance of meeting each target in each season t, to release.

constraint	10<=r<=40	
TS	20	
TR	25	
Min TSD		
TSD	(TS-s_t)^2	+ (TR-Rt)^2

Release

Q1	10		Final storage			
			0	10	20	30
Season 1	Initial storage	0	10	NA	NA	NA
		10	20	10	NA	NA
		20	30	20	10	NA
		30	40	30	20	10

Q2	50		Final storage			
			0	10	20	30
Season 2	Initial storage	0	10	NA	NA	NA
		10	20	10	NA	NA
		20	30	20	10	NA
		30	40	30	20	10

Q3	20		Final storage			
			0	10	20	30
Season 3	Initial storage	0	20	10	0	-
		10	30	20	10	0
		20	40	30	20	10
		30	-	40	30	20

TSD

Q3	20		Final storage			
			0	10	20	30
Season 3	Initial storage	0	425	625	NA	NA
		10	125	125	325	NA
		20	225	25	25	225
		30	NA	325	125	125

Q2	50		Final storage			
			0	10	20	30
Season 2	Initial storage	0	NA	625	425	425
		10	NA	NA	325	125
		20	NA	NA	NA	225
		30	NA	NA	NA	NA

Q1	10		Final storage			
			0	10	20	30
Season 1	Initial storage	0	625	NA	NA	NA
		10	125	325	NA	NA
		20	25	25	225	NA
		30	325	125	125	325

Backward

			Final storage						
			0	10	20	30	Ft	St+1	Rt
Season 3 n=1	Initial storage	0	425	625	NA	NA	425	0	20
		10	125	125	325	NA	125	0, 10	20, 30
		20	225	25	25	225	25	10,20	20, 30
		30	NA	325	125	125	125	20, 30	20, 30

			Final storage						
			0	10	20	30	Ft	St+1	Rt
Season 2 n=2	Initial storage	0	NA	750	450	550	450	20	30
		10	NA	NA	350	250	250	30	30
		20	NA	NA	NA	350	350	30	40
		30	NA	NA	NA	NA	NA	NA	NA

			Final storage						
			0	10	20	30	Ft	St+1	Rt
Season 1 n=3	Initial storage	0	1075	NA	NA	NA	1075	0	10
		10	575	575	NA	NA	575	0,10	10,20
		20	475	275	575	NA	275	10	20
		30	775	375	475	NA	375	10	30

			Final storage						
			0	10	20	30	Ft	St+1	Rt
Season 3 n=4	Initial storage	0	1500	1200	NA	NA	1200	10	10
		10	1200	700	600	NA	600	20	10
		20	1300	600	300	600	300	20	20
		30	NA	900	400	500	400	20	30

			Final storage									
			0	10	20	30	Ft	St+1	Rt			
Season 2 n=5	Initial storage	0	NA	1225	725	800	725	20	30	275	F[season=2, n=5]	F[season=2, n=2]
		10	NA	NA	625	525	525	30	30	275		
		20	NA	NA	NA	625	625	30	40	275		
		30	NA	NA	NA	NA	NA	NA	NA	NA		

			Final storage									
			0	10	20	30	Ft	St+1	Rt			
Season 1 n=6	Initial storage	0	1350	NA	NA	NA	1350	0	10	275	F[season=1, n=6]	F[season=1, n=3]
		10	850	850	NA	NA	850	0,10	10,20	275		
		20	750	550	850	NA	550	10	20	275		
		30	1050	650	750	NA	650	10	30	275		

			Final storage									
			0	10	20	30	Ft	St+1	Rt			
Season 3 n=7	Initial storage	0	1775	1475	NA	NA	1475	10	10	275	F[season=3, n=7]	F[season=3,n =4]
		10	1475	975	875	NA	875	20	10	275		
		20	1575	875	575	875	575	20	20	275		
		30	NA	1175	675	775	675	20	30	275		

When n=7, not only release R_t and final storage volume St+1 is the same in each succeeding year but also the difference $F_t^{n+3}(s_t, Q_t) - F_t^n(s_t, Q_t)$ is a constant, 275 for all S_t and t. This value (275) is the minimum annual sum of squared deviations that can be obtained by following the derived sequential operating policy.

[Table 1] The discrete steady-state reservoir operating policy as computed for problem

Initial Storage	Release Season t			Final storage volume Season t		
S	1	2	3	1	2	3
0	10	30	10	0	20	10
10	10, 20	30	10	0, 10	30	20
20	20	40	20	10	30	20
30	30	-	30	10	-	20

[R code for this problem]

```r
#constraint 10<=r<=40
#objective function: min(TSD)
#TSD=(TS-s_t)^2+(TR-Rt)^2
#Rt = s_t + Qt - s_t+1

TS=array(20,c(4,4))
TR=array(25,c(4,4))
ini_S=c(0, 10, 20, 30)
s_t=array(ini_S, c(4,4))
s_t1=array(c(rep(0,4), rep(10,4),rep(20,4), rep(30,4)), c(4,4))

Qt_1=array(c(rep(10,16)), c(4,4))
Qt_2=array(c(rep(50,16)), c(4,4))
Qt_3=array(c(rep(20,16)), c(4,4))

Rt1= s_t+Qt_1-s_t1
Rt2= s_t+Qt_2-s_t1
Rt3= s_t+Qt_3-s_t1

Rt1[Rt1<10 | Rt1>40]<-NA
Rt2[Rt2<10 | Rt2>40]<-NA
Rt3[Rt3<10 | Rt3>40]<-NA

Rt1
Rt2
Rt3

#Calculate TSD
TSD1<-array(0, c(4,4))
for(i in 1:4){
  for(j in 1:4){
    TSD1[i,j] <-(TS[i,j]-s_t[i,j])^2 + (TR[i,j]-Rt1[i,j])^2
  }
}
TSD1

TSD2<-array(0, c(4,4))
for(i in 1:4){
  for(j in 1:4){
    TSD2[i,j] <-(TS[i,j]-s_t[i,j])^2 + (TR[i,j]-Rt2[i,j])^2
  }
}
TSD2

TSD3<-array(0, c(4,4))
for(i in 1:4){
  for(j in 1:4){
    TSD3[i,j] <-(TS[i,j]-s_t[i,j])^2 + (TR[i,j]-Rt3[i,j])^2
  }
}
```

```
TSD3

min(TSD3[1,], na.rm=TRUE)
min(TSD2[1,], na.rm=TRUE)
min(TSD1[1,], na.rm=TRUE)

TSD=list(TSD3, TSD2, TSD1)
Rt=list(Rt3, Rt2, Rt1)
TSD[[3]]

#Backward
ft_s<-list()
s_s<-list()
s_rt<-list()
for(i in 1:4){

  ft= min(TSD3[i,], na.rm=TRUE)
  s_loc=which(TSD3[i,]==ft)
  s_tt=ini_S[s_loc]
  r_tt=Rt3[i,s_loc]

  ft_s[[i]]=ft
  s_s[[i]]=c(s_tt)
  s_rt[[i]]=c(r_tt)

}
ft_result=list(c(),c(),c())
result=data.frame(cbind(ft_s),cbind(s_s), cbind(s_rt) )
a= array(rep(result[,1], each=4), dim=c(4,4))
ft_result[[1]][[1]] = result

#Backward recursive algorithm
for (i in 2:100){
  #i=2
  j = i%%3
  jj = (i-1)%%3

  if (j == 0){
    j = j+3
  }

  if (jj == 0){
    jj = jj+3
  }

  ft_model = TSD[[j]]
  ft_Rt = Rt[[j]]
  ll = length(ft_result[[j]])
  lll = length(ft_result[[jj]])
  before_re=ft_result[[jj]][[lll]]
```

```
ft_l = list()
release_l = list()
storage_l = list()

for(k in 1:4){
  #k=1
  if (is.na(ft_model[k,1]) & is.na(ft_model[k,2]) & is.na(ft_model[k,3]) & is.na(ft_model[k,4])){
    ft_l[[k]] = NA
    release_l[[k]] = NA
    storage_l[[k]] = NA
    next
  }

  for (l in 1:4){
    #l=2
    if (is.na(ft_model[1,l]) & is.na(ft_model[2,l]) & is.na(ft_model[3,l]) & is.na(ft_model[4,l])){
      next
    }
    if (is.na(ft_model[k,l])){
      next
    }
    if (is.na(before_re[,1][[l]])){
      ft_model[k,l] = NA
      next
    }
    ft_model[k,l] = ft_model[k,l] + before_re[,1][[l]]

  }

  fts = min(ft_model[k,],na.rm=T)
  s_loc = which(ft_model[k,]==fts)
  s_rtt = ft_Rt[k,s_loc]
  s_s = ini_S[s_loc]

  ft_l[[k]] = fts
  release_l[[k]] = c(s_rtt)
  storage_l[[k]] = c(s_s)
}

ft.r = data.frame(cbind(ft_l), cbind(storage_l), cbind(release_l))

ft_result[[j]][[(ll+1)]] = ft.r

if (ll>2){
  a = unlist(ft_result[[j]][[(ll-1)]][,1]) - unlist(ft_result[[j]][[ll]][,1])
  b = unlist(ft_result[[j]][[ll]][,1]) - unlist(ft_result[[j]][[(ll+1)]][,1])
  if (a[1]==b[1] & a[2]==b[2] & a[3]==b[3] & a[4]==b[4]) break
}
}
```

D. 4.18- Dynamic programming

The following table provides estimates for the recent values of the costs of additional wastewater treatment plant capacity needed at the end of each 5-year period for the next 20 years. Find the capacity expansion schedule that minimizes the present values of the total future costs. If there is more than one least cost solution, indicate which one you think is better, and why?

		Discounted cost of additional capacity					Total required capacity at end of period
		Units of additional capacity					
Period years		2	4	6	8	10	
1	1–5	12	15	18	23	26	2
2	6–10	8	11	13	15		6
3	11–15	6	8				8
4	16–20	4					10

2. Solve the following LP Problem in Excel and in R:

A. Attach you R code.

```r
#install.packages('lpSolve')
library('lpSolve')
# Max. z=10x1 + 8x2 + 9x3
# 목적함수
z<-c(10,8,9)
# 제약조건 좌변
A<-matrix(c(2,3,1,
            5,6,6,
            1,1,1,
            0,1,0,
            1,0,0,
            0,0,1), nrow=6, byrow=TRUE)
# 제약조건 우변
B<-c(1000, 2400, 700, 160, 0,0)
# 제약식 부등호
dir<-c("<=", "<=", "<=", ">=", ">=", ">=")
# 최적해
opt<-lp(direction="max",
        objective.in = z,
        const.mat = A,
        const.dir = dir,
        const.rhs = B,
        all.int = T)
# 결과
opt
# 결정변수
opt$solution
```

B. Compare your results obtained from Excel and R. Are they identical?

Maximize	10	8	9			
	2	3	1	1000	<=	1000
	5	6	6	2400	<=	2400
	1	1	1	440	<=	700
	1			240	>=	0
		1		160	>=	160
			1	40	>=	0
	240	160	40	4040		

(Results obtained from Excel)

```
> # 결과
> opt
Success: the objective function is 4040
> # 결정변수
> opt$solution
[1] 240 160  40
```

(Results obtained from R)

They are identical.

3. Solve the following DP problem in R:

The inflows to a reservoir having a total capacity of four units of water are 1, 2, 1, and 2 units, respectively, for the four seasons in a year. For convenience, the amount of water is counted only be integer units of water. Thereby, the releases from the reservoir for water supply to a city and farms are sold for $2000 for the first unit of release, $1500 for second unit, $1000 for the third unit, and $500 for the fourth unit. When the reservoir is full and spills one unit of water, a minor flood with result in a damage of $1500. When the spill reaches 2 units, a major flood will result in a damage of $4000. Determine the operation policy for maximum annual returns by dynamic programming using a backward algorithm, considering any amount of storage at the end of the year.

A. First, set the benefit function of your problem. How would you set your benefit function?

$NB_t(S_t,S_{t+1}, R_t)$ = Total return – Total cost

Release from the reservoir during a season results in the following benefits which are same for all the four seasons.

Release	Benefits	
1	2000=	$2000
2	2000+1500=	$3500
3	2000+1500+1000=	$4500
4	2000+1500+1000+500=	$5000
5	2000+1500+1000+500-1500=	$3500
6	2000+1500+1000+500-4000=	$1000

B. What is the governing equation of your problem?

$S_{t+1} = S_t + Q_t - R_t$

where S_{t+1} is final storage of season t (the initial storage of t+1 season) , S_t is initial storage, Q_t is the inflow, and R_t is the release of season n.

C. What is the DP backward recursive equation for this example? Your equations should be different for seasons 1-3, and season 4.

For season 4, $F_t^n(S_t) = \text{Maximum}\{NB_t(S_t, R_t)\}$

for all $0 \leq S_t \leq 4$, $S_t + Q_t - R_t \leq 4$

For season 1-3, $F_t^n(S_t) = \text{Maximum}\{NB_t(S_t, R_t) + F_{t+1}^{n-1}(S_{t+1})\}$

for all $0 \leq S_t \leq 4$, $S_t + Q_t - R_t \leq 4$

D. Solve your problem in R. You may solve your problem first using Excel first.

[Table 2] The discrete steady-state reservoir operating policy as computed for problem

Initial Storage	Release Season t				Final storage volume Season t			
S	1	2	3	4	1	2	3	4
0	1	1, 2	1	1, 2	0	0, 1	0	0, 1
1	1, 2	1, 2	1, 2	1, 2	0, 1	1, 2	0, 1	1, 2
2	1, 2	1, 2	1, 2	1, 2	1, 2	2, 3	1, 2	2, 3
3	1, 2	1, 2	1, 2	1, 2	2, 3	3, 4	2, 3	3, 4
4	1, 2	2	1, 2	2	3, 4	4	3, 4	4

Season 4

T=4	N=13			N=9			
	(1)	(2) St+1	(3)Rt	(4)ft	(5)St+1	(6)Rt	(1)-(4)
0	36500	0, 1	1, 2	25500	0, 1	1, 2	11000
1	38000	1, 2	1, 2	27000	1, 2	1, 2	11000
2	39500	2, 3	1, 2	28500	2, 3	1, 2	11000
3	41000	3, 4	1, 2	30000	3, 4	1, 2	11000
4	42500	4	2	31500	4	2	11000

Season 3

T=3	N=14			N=10			
	(1)	(2) St+1	(3)Rt	(4)ft	(5)St+1	(6)Rt	(1)-(4)
0	38500	0	1	27500	0	1	11000
1	40000	0, 1	1, 2	29000	0, 1	1, 2	11000
2	41500	1, 2	1, 2	30500	1, 2	1, 2	11000
3	43000	2, 3	1, 2	32000	2, 3	1, 2	11000
4	44500	3, 4	1, 2	33500	3, 4	1, 2	11000

Season 2

T=2	N=15			N=11			
	(1)	(2) St+1	(3)Rt	(4)ft	(5)St+1	(6)Rt	(1)-(4)
0	42000	0, 1	1, 2	31000	0, 1	1, 2	11000
1	43500	1, 2	1, 2	32500	1, 2	1, 2	11000
2	45000	2, 3	1, 2	34000	2, 3	1, 2	11000
3	46500	3, 4	1, 2	35500	3, 4	1, 2	11000
4	48000	4	2	37000	4	2	11000

Season 1

T=1	N=12			N=8			
	(1)	(2) St+1	(3)Rt	(4)ft	(5)St+1	(6)Rt	(1)-(4)
0	33000	0	1	22000	0	1	11000
1	34500	0, 1	1, 2	23500	0, 1	1, 2	11000
2	36000	1, 2	1, 2	25000	1, 2	1, 2	11000
3	37500	2, 3	1, 2	26500	2, 3	1, 2	11000
4	39000	3, 4	1, 2	28000	3, 4	1, 2	11000

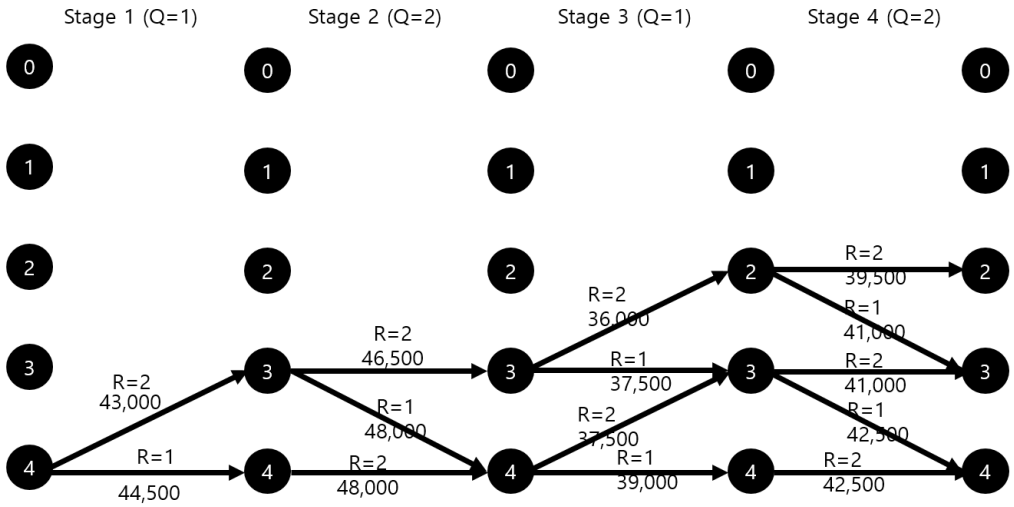
[R code for this problem]

```r
#set Qt, St, St+1, Rt
ini_S=c(0, 1, 2, 3, 4)
s_t=array(ini_S, c(5,5))
s_t1=array(c(rep(0,5), rep(1,5),rep(2,5), rep(3,5), rep(4,5)), c(5,5))

Qt_1=array(c(rep(1,25)), c(5,5))
Qt_2=array(c(rep(2,25)), c(5,5))
Qt_3=array(c(rep(1,25)), c(5,5))
Qt_4=array(c(rep(2,25)), c(5,5))

Rt1= s_t+Qt_1-s_t1
Rt2= s_t+Qt_2-s_t1
Rt3= s_t+Qt_3-s_t1
Rt4= s_t+Qt_4-s_t1

Rt1[Rt1<0 ]<-NA
Rt2[Rt2<0 ]<-NA
Rt3[Rt3<0 ]<-NA
Rt4[Rt4<0 ]<-NA

Rt1
Rt2
Rt3
Rt4

Rt=list(Rt1, Rt2, Rt3, Rt4)

#Benefit function
bf=c(0,2000,3500,4500,5000,3500,1000)
NBF=list()

for (i in 1:4){

  RT = Rt[[i]]
  NB = array(NA, c(5,5))

  for (j in 1:5){

    for (k in 1:5){

      w = RT[j,k]
      benefit = bf[(w+1)]
      NB[j,k] = benefit

    }

  }
  NBF[[i]] = NB
}
NBF
```

```
51
52  #Backward for first
53  back_NBF=list(NBF[[4]],NBF[[3]], NBF[[2]],NBF[[1]] )
54  back_RT=list(Rt4,Rt3, Rt2, Rt1)
55
56  NBF4=NBF[[4]]
57  ft_s<-list()
58  s_s<-list()
59  s_rt<-list()
60  for(i in 1:5){
61
62    ft= max(NBF4[i,], na.rm=TRUE)
63    s_loc=which(NBF4[i,]==ft)
64    s_tt=ini_S[s_loc]
65    r_tt=Rt4[i,s_loc]
66
67    ft_s[[i]]=ft
68    s_s[[i]]=c(s_tt)
69    s_rt[[i]]=c(r_tt)
70
71  }
72  ft_result=list(c(),c(),c(), c())
73  result=data.frame(cbind(ft_s),cbind(s_s), cbind(s_rt) )
74  a= array(rep(result[,1], each=5), dim=c(5,5))
75  ft_result[[1]][[1]] = result
76
77  #backward
78  for (i in 2:100){
79    #i=2
80    j = i%%4
81    jj = (i-1)%%4
82
83    if (j == 0){
84      j = j+4
85    }
86
87    if (jj == 0){
88      jj = jj+4
89    }
90
91    ft_model = back_NBF[[j]]
92    back_rt = back_RT[[j]]
93    ll = length(ft_result[[j]])
94    lll = length(ft_result[[jj]])
95    before_re = ft_result[[jj]][[lll]]
96
97    ft_l = list()
98    release_l = list()
99    storage_l = list()
100
```

```
  for (k in 1:5){
    #k=1
    if (is.na(ft_model[k,1]) & is.na(ft_model[k,2]) & is.na(ft_model[k,3]) & is.na(ft_model[k,4])){
      ft_l[[k]] = NA
      release_l[[k]] = NA
      storage_l[[k]] = NA
      next
    }

    for (l in 1:5){
      #l=2
      if (is.na(ft_model[1,l]) & is.na(ft_model[2,l]) & is.na(ft_model[3,l]) & is.na(ft_model[4,l])){
        next
      }
      if (is.na(ft_model[k,l])){
        next
      }
      if (is.na(before_re[,1][[l]])){
        ft_model[k,l] = NA
        next
      }
      ft_model[k,l] = ft_model[k,l] + before_re[,1][[l]]

    }

    fts = max(ft_model[k,],na.rm=T)
    s_loc = which(ft_model[k,]==fts)
    s_rtt = back_rt[k,s_loc]
    s_s = ini_S[s_loc]

    ft_l[[k]] = fts
    release_l[[k]] = c(s_rtt)
    storage_l[[k]] = c(s_s)
  }

  ft.r = data.frame(cbind(ft_l), cbind(storage_l), cbind(release_l))

  ft_result[[j]][[(ll+1)]] = ft.r

  if (ll>4){
    a = unlist(ft_result[[j]][[(ll-1)]][,1]) - unlist(ft_result[[j]][[ll]][,1])
    b = unlist(ft_result[[j]][[ll]][,1]) - unlist(ft_result[[j]][[(ll+1)]][,1])
    if (a[1]==b[1] & a[2]==b[2] & a[3]==b[3] & a[4]==b[4] & a[5]==b[5]) break
  }
}
```

E. Assume that the starting storage (S0, before S1) is 4. What are the optimal release decisions during the four seasons? There should be multiple optimal release decisions. Name all and compute each path's annual return values.



4. In R, replicate the reservoir model on Boryeong Dam by you built on Assignment #1 with actual zone-based hedging rules provided below:

A. First, interpolate daily values with the provided values. You should use linear interpolation method in R. You do not have to include your interpolation results.

B. Assume that at the 탄력 공급 stage, water supply for M&I(생공용수) water is 3.0m3/s instead of 2.9m3/s. Conduct a reservoir simulation using the daily 용수공급조정기준 you interpolated in A. Your model should consider the maximum and minimum constraints.

[R code for this problem]

```r
#install.packages("readxl")
library(readxl)
#install.packages('zoo')
library(zoo)
#install.packages("ggplot2")
library(ggplot2)
library(lubridate)

setwd("C:/Users/User/Dropbox/2022수시템")
inflow=read_excel("hw2-4.xlsx", sheet = 3, col_names = T)
head(inflow)

demand = read_excel("hw2-4.xlsx", sheet = 2, col_names = T)
head(demand)

interpo=read_excel("hw2-4.xlsx", sheet = 1, col_names = T)
head(interpo)

#4-A. interpolation

date=read_excel("hw2-4.xlsx", sheet = 5, col_names = T)
head(date)
zc <- zoo(c(interpo$관심), interpo$date)    # low freq
zs <- zoo(c(date$관심), date$date)  # high freq
# Merge series into one object
z <- merge(zs,zc)
# Interpolate calibration data (na.spline could also be used)
z$zc <- na.approx(z$zc, rule=2)
# Only keep index values from sample data
Z <- z[index(zs),]
head(Z)

zc2 <- zoo(c(interpo$주의), interpo$date)    # low freq
zs2 <- zoo(c(date$주의), date$date)  # high freq
z2 <- merge(zs2,zc2)
z2$zc2 <- na.approx(z2$zc2, rule=2)
Z2 <- z2[index(zs2),]
head(Z2)

zc3 <- zoo(c(interpo$경계), interpo$date)    # low freq
zs3 <- zoo(c(date$경계), date$date)  # high freq
z3 <- merge(zs3,zc3)
z3$zc3 <- na.approx(z3$zc3, rule=2)
Z3 <- z3[index(zs3),]
head(Z3)

zc4 <- zoo(c(interpo$심각), interpo$date)    # low freq
zs4 <- zoo(c(date$심각), date$date)  # high freq
z4 <- merge(zs4,zc4)
z4$zc4 <- na.approx(z4$zc4, rule=2)
```

```r
Z4 <- z4[index(zs4),]
head(Z4)

rule<-cbind( Z$zc, Z2$zc2, Z3$zc3, Z4$zc4)
head(rule)
dim(rule)
rule<-data.frame(rule)
com_rule<-rbind(rule[c(265:365),], rule[c(1:264),])
head(com_rule)

#4-B. Simulation
demand1<-demand$합

demand2.1<-array(rep(3*60*60*24/1000000,12),c(12,1))
demand2_1<-cbind(demand2.1, demand$농업, demand$하천)
demand2<-rowSums(demand2_1)  #합해야함

demand3_1<-cbind(demand$생공, demand$농업, array(0,c(12,1)))
demand3<-rowSums(demand3_1)

demand3_1[c(4:6),2]*0.8
demand3_1[c(7:9),2]*0.7
demand4_1_agri<-c(demand3_1[c(1:3),2], demand3_1[c(4:6),2]*0.8,demand3_1[c(7:9),2]*0.7,c(demand3_1[c(10:12),2]))
demand4_1<-cbind(demand$생공,demand4_1_agri,array(0,c(12,1)) )
demand4<-rowSums(demand4_1)

demand5_1<-cbind( demand$생공*0.8,demand4_1_agri,array(0,c(12,1)) )
demand5<-rowSums(demand5_1)

demand_<-cbind(demand2, demand1, demand3, demand4, demand5)
demand<-data.frame(demand_)
demand<-rbind(demand_[10:12,],demand_[1:9,])
dim(demand)

ini_S = 40.025
Smax = 108.7
Smin = 6.2
storage = array(NA, c(365))
storage[1] = inflow[1,2] - demand_[10,4] + ini_S

for (i in 2:365){
  for(j in 1:12){

    com_rules = com_rule[i,1:4]
    inflow_ = inflow[i,2]
    storage_1 = storage[(i-1)]


```

```
    if (storage_1 > com_rules[1]){
      Rt = demand_[j,1]
      storage[i] = max(Smin, as.numeric(inflow_ + storage_1 - Rt))

    }else if (com_rules[1] > storage_1 & storage_1 > com_rules[2]){
      Rt = demand_[j,2]
      storage[i] =  max(Smin, as.numeric(inflow_ + storage_1 - Rt))

    }else if (com_rules[2] > storage_1 & storage_1 > com_rules[3]){
      Rt = demand_[j,3]
      storage[i] =  max(Smin, as.numeric(inflow_ + storage_1 -Rt))

    }else if (com_rules[3] >storage_1 & storage_1 > com_rules[4]){
      Rt = demand_[j,4]
      storage[i] =  max(Smin, as.numeric(inflow_ + storage_1 -Rt))
    }

    else if (com_rules[4] > storage_1){
      Rt = demand_[j,5]
      storage[i] =  max(Smin, as.numeric(inflow_ + storage_1 -Rt))
    }
  }
}


act_release = array(NA, c(365))
act_release[1] = demand_[1,4]


for (i in 2:365){
  act_release[i] = as.numeric(storage[(i-1)] + inflow[i,2] - storage[i])
}

with=array(storage, c(365,1))
without=read_excel("hw2-4.xlsx", sheet = 4, col_names = T)
head(without)
dates=as.POSIXct(without$Date)
without1=as.numeric(without$Storage)
with=as.numeric(with)
data<-data.frame(dates, without1, with)

ggplot(data=data, aes(x=dates))+xlab("Time")+ylab("Storage(MCM)")+
  geom_line(aes(y = without1, colour = "without hedging")) +
  geom_line(aes(y = with, colour = "with hedging")) +
  scale_colour_manual("",
                      breaks = c("without hedging", "with hedging"),
                      values = c("red", "blue"))

without2=as.numeric(without$Release)
with2=as.numeric(array(act_release, c(365,1)))
data2<-data.frame(dates, without2, with2)

ggplot(data=data2, aes(x=dates))+xlab("Time")+ylab("Release(MCM)")+
  geom_line(aes(y = without2, colour = "without hedging")) +
  geom_line(aes(y = with2, colour = "with hedging")) +
  scale_colour_manual("",
                      breaks = c("without hedging", "with hedging"),
                      values = c("red", "blue"))

```

C. Check if your model meets the annual mass balance equation (Total Inflow + Starting Storage = Total Actual Outflow + Final Storage). What is the sum of the left-hand side equation? Right-hand side of the equation? (They should be very much similar to each other)

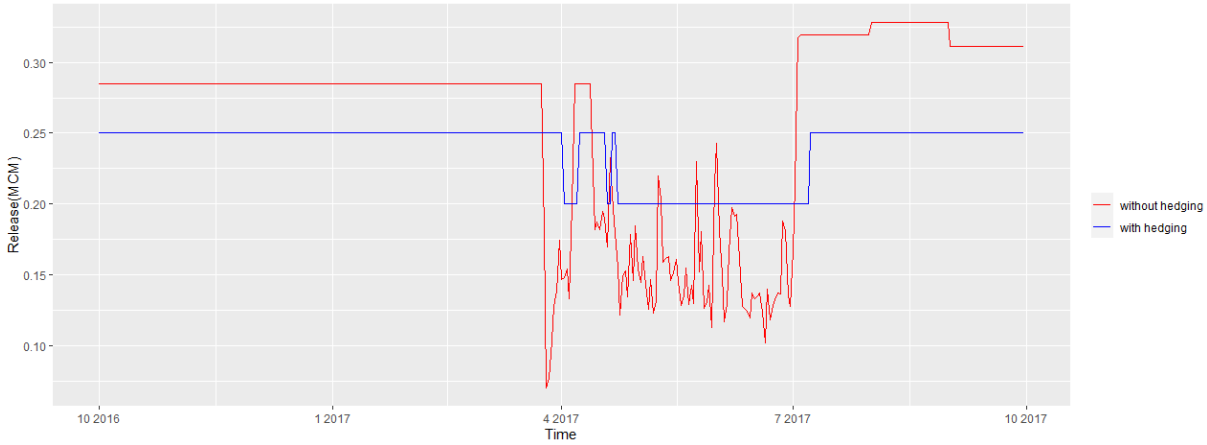
```
158  #4-C
159  LHS = ini_S + sum(inflow[,2])
160  RHS = sum(as.numeric(act_release))+as.numeric(storage[365])
161
162  LHS
163  RHS
164
34:36  (Top Level)
```

```
Console  Terminal  Jobs
R 4.1.1 · C:/Users/User/Dropbox/2022수시텀/
> #4-C
> LHS = ini_S + sum(inflow[,2])
> RHS = sum(as.numeric(act_release))+as.numeric(storage[365])
>
> LHS
[1] 120.8349
> RHS
[1] 120.8349
>
```

LHS = RHS = 120.8349

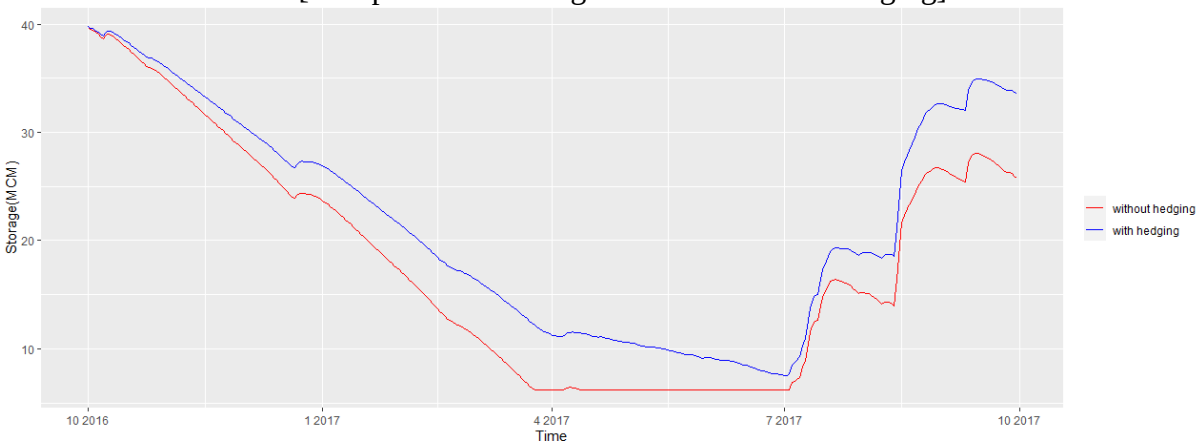
D. Finally, compare your results with results from no hedging. What is the effect of hedging? Qualitatively? Quantitively?

[Comparison of release with and without hedging]



Due to the hedging effect, the amount of discharge can be controlled from the end of March. When we look at the graph below, it can be operated more effectively by delaying the period of falling to the minimum storage of 6.2.

[Comparison of storage with and without hedging]



Due to the hedging effect, when a sudden drought occurs from the point of view of consumers or suppliers, it can be stable because the rate or slope of the decrease in storage is gentle. Also, since the amount of storage decreased and then ascended, it will be possible to recover from difficulties quickly.